Stock-Market

End-to-End Autonomous Trading System

A fully autonomous quantitative trading system running on a personal computer against Interactive Brokers. The pipeline covers SEC data ingestion, high-speed IBKR price collection, alpha factor engineering, XGBoost prediction, multi-metric backtesting, LLM-assisted macro filtering, and live bracket-order execution. A single PowerShell scheduler fires twice daily. No human intervention required.

~4,200	9	20+	Live
EQUITIES TRACKED	PIPELINE STAGES	RISK & PERF METRICS	IBKR PRODUCTION

VIEW SOURCE ON GITHUB
(click to open repo)

GITHUB.COM/JONISHERE242
(click me)

// 01. OVERVIEW

What This System Does and Why

Most retail quantitative systems stop at backtesting. This one didn't. Every component from data ingestion to live order submission is written, owned, and running in production. Each stage does one thing and passes its output forward. Complexity is at the system level, not buried inside any individual module. That makes each piece independently testable and replaceable, which matters a lot when something breaks at 9:45am on a trading day.(or IBKR resets my password 5 minutes before market open)

☐ WHAT MAKES THIS NON-TRIVIAL

The system filters ~10,000 SEC-registered securities down to ~4,200 investable equities, pulls daily price history for all of them via a custom IBKR async connection pool, engineers 50+ alpha features per ticker across all CPU cores, predicts next-day direction with a calibrated XGBoost classifier, validates every strategy idea through a backtester tracking 20+ risk metrics, runs a news scan via Claude in research mode, and submits four-legged bracket orders with automated downside protection. All of this finishes before market open each morning.

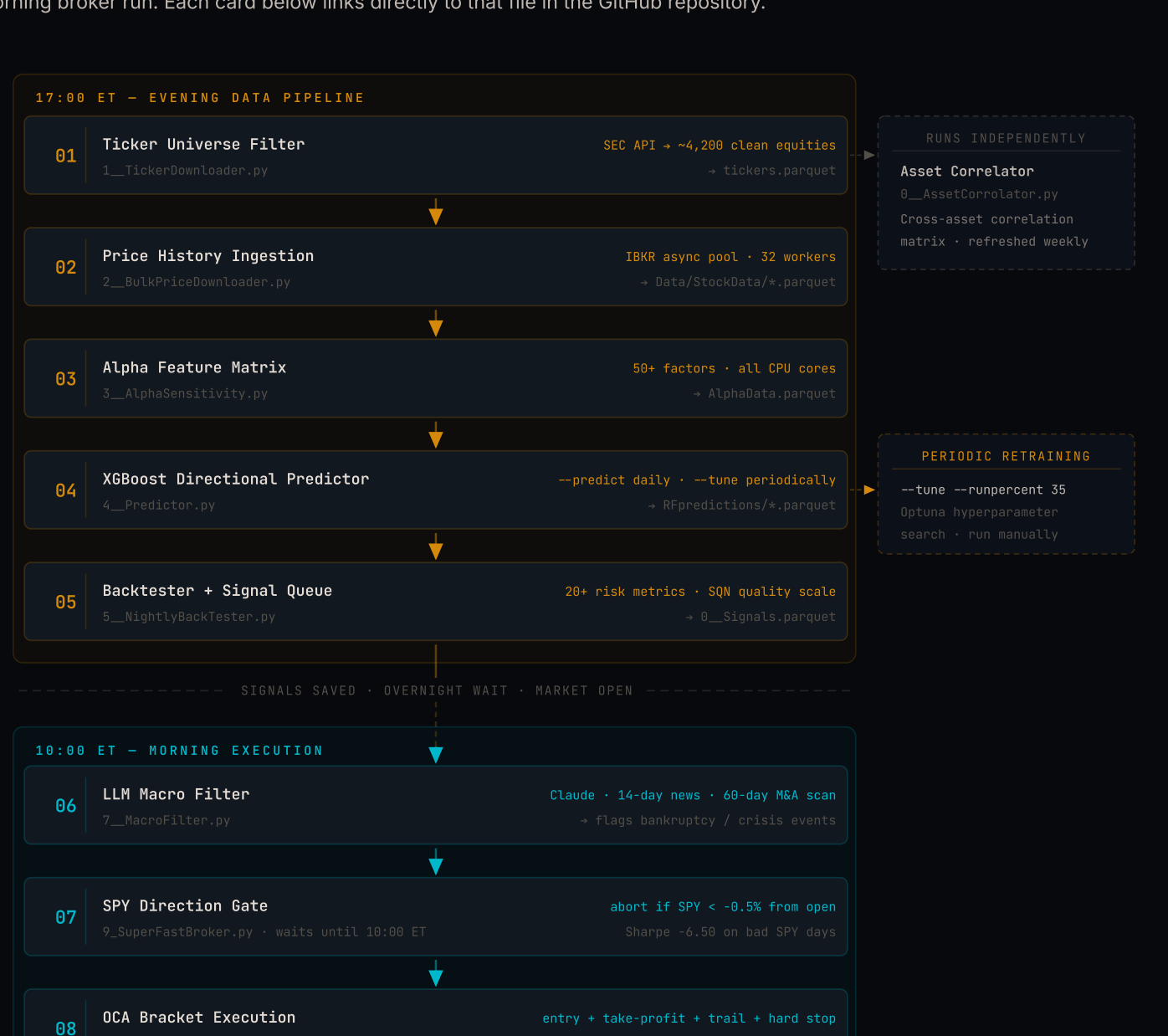
☐ WHY LONG-ONLY EQUITIES, NO ETFS, NO REITS, NO BANKS

The prediction target is next-day directional move for individual operating companies. ETFs introduce index replication noise. REITs are driven by cap rates and distribution schedules. Banks have opaque balance sheets where standard momentum and mean-reversion signals behave erratically. SPACs, leveraged products, and shell companies are rejected at the ticker universe stage before any price data is even pulled. The filter keeps the training set in a regime where the features are actually informative. Somewhat ironic given the context of this document, but that's what the data said.

// 02. PIPELINE

Nine-Stage Execution Pipeline

Orchestrated by `trading_system.ps1`, which fires at 17:00 for the evening data run and again at 07:00 for the morning broker run. Each card below links directly to that file in the GitHub repository.


17:00 ET — EVENING DATA PIPELINE
01 Ticker Universe Filter
1__TickerDownloader.py
SEC API → ~4,200 clean equities
→ tickers.parquet
02 Price History Ingestion
2__BulkPriceDownloader.py
IBKR async pool · 32 workers
→ Data/StockData/*.parquet
03 Alpha Feature Matrix
3__AlphaSensitivity.py
50+ factors · all CPU cores
→ AlphaData.parquet
04 XGBoost Directional Predictor
4__Predictor.py
--predict daily · --tune periodically
→ RFpredictions/*.parquet
05 Backtester + Signal Queue
5__NightlyBackTester.py
20+ risk metrics · SQN quality scale
→ 0__Signals.parquet
RUNS INDEPENDENTLY
Asset Correlator
0__AssetCorrolator.py
Cross-asset correlation matrix · refreshed weekly
PERIODIC RETRAINING
--tune --runpercent 35
Optuna hyperparameter search · run manually
SIGNALS SAVED · OVERNIGHT WAIT · MARKET OPEN
10:00 ET — MORNING EXECUTION
06 LLM Macro Filter
7__MacroFilter.py
Claude · 14-day news · 60-day M&A scan
→ flags bankruptcy / crisis events
07 SPY Direction Gate
9_SuperFastBroker.py · waits until 10:00 ET
abort if SPY < -0.5% from open
Sharpe -6.50 on bad SPY days
08 OCA Bracket Execution
9_SuperFastBroker.py · IBKR Adaptive Algo
entry + take-profit + trail + hard stop
→ live position opened

click any stage to open that file on GitHub

STAGE 0 → GitHub
Scheduler / Orchestrator
PowerShell runner fires at 17:00 and 07:00. Executes the full pipeline sequentially with 20-second RAM flush pauses between stages. Timestamped logging to disk on every step.
trading_system.ps1

STAGE 1 → GitHub
Ticker Universe Construction
Pulls the live SEC company registry, applies 30+ keyword and regex filters, removes OTC and CBOE listings. Outputs ~4,200 clean equity tickers as a timestamped parquet.
1__TickerDownloader.py

STAGE 2 → GitHub
High-Speed IBKR Price Ingestion
Async connection pool (deque-backed, up to 32 workers) pulls incremental daily OHLCV bars from TWS at ~20 tickers/second. Retry logic, health tracking, and progress persistence across restarts.
2__PriceDownloader.py

STAGE 3 → GitHub
Alpha Factor Engineering
50+ features per ticker: rolling beta vs SPY/QQQ/IWM/DIA, VIX regime features, Kalman-filtered trend, FFT spectral decomposition, visibility graph metrics, entropy, skew, kurtosis, DVAMR probability. ProcessPoolExecutor saturates all CPU cores.
3__AlphaSensitivity.py

STAGE 4 → GitHub
XGBoost Directional Predictor
Binary classifier (up vs not-up next day). Trained on temporal splits with 5-day embargo gaps. AUC-PR early stopping. Isotonic calibration converts raw scores to well-calibrated 0–1 probabilities. 24 parallel trees, 32 threads.
4__Predictor.py

STAGE 5 → GitHub
Research Backtesting Environment
Backtrader-based simulator with exact IBKR commission model ($0.0035/share, $0.35 min). Computes 20+ metrics including Sharpe, Sortino, Calmar, Omega, Ulcer Index, VaR, CVaR, and a custom SQN quality scale. Each metric color-coded from red to green to Unicorn blue.
5__NightlyBackTester.py

STAGE 6 → GitHub
LLM Macro Filter
Claude (Sonnet) in research mode searches recent news per signal ticker. Flags bankruptcy, active M&A, geopolitical crisis, or catastrophic events that regex cannot reliably catch. Cached results with resume-on-failure support.
7__MacroFilter.py

STAGE 7 → GitHub
Live Execution Engine
Waits until 10:00 ET (EDA-derived: +256% Sharpe vs market open). Gates on SPY direction. Submits four-legged OCA bracket orders: entry, take-profit, trailing stop, hard stop. All exit legs share one OCA group so whichever fills first cancels the others automatically.
9_SuperFastBroker.py

// 03. STAGE 1: TICKER UNIVERSE

Building a Clean Equity Universe from SEC Data

Every night the system pulls the SEC's `company_tickers_exchange.json` endpoint, the same registry regulators use. Raw output contains 10,000+ entries: operating companies, SPACs, ETFs, leveraged products, shell companies, funds, warrants. The filtering stage cuts that down to ~4,200 investable equities using two passes: keyword rejection first, then regex patterns for anything keywords miss.

☐ LAYER 1: INSTANT-REJECT KEYWORDS

Any company whose name contains one of these words is rejected before further processing. This catches the vast majority of non-equity garbage in one O(n) pass:

`acquisition` `spac` `etf` `fund` `reit` `warrant` `preferred` `holdings` `vehicle` `ventures` `income` `blockchain` `metaverse` `crypto` `merger` `bitcoin`

☐ LAYER 2: REGEX PATTERN FILTERS

A second pass applies regex patterns for cases keywords miss. Catches leveraged products, series/class designations, numbered SPAC variants, and known fund provider names:

`vanguard` `ishares` `spdr` `blackrock` `\d+x\s` `leveraged` `inverse` `series [a-z]` `class [a-z]` `shell company` `blank check` `roman numerals after corp/inc`

☐ WHY THIS MATTERS FOR SIGNAL QUALITY

A directional equity predictor trained on a contaminated universe leaks ETF replication behavior, SPAC announcement dynamics, and fund distribution effects into the feature space. These have fundamentally different return drivers than operating companies. Filtering hard at the universe stage, before any price data is downloaded or any feature is computed, keeps the downstream feature matrix consistent across market regimes. The cost is a few false positives on legitimate companies with unfortunate names. That tradeoff is worth it.

// 04. STAGE 2: PRICE INGESTION

Async IBKR Connection Pool: ~20 Tickers/Second

Pulling daily price history for 4,200 tickers via Interactive Brokers TWS is a rate-limiting problem. IBKR throttles requests per connection, but allows multiple simultaneous connections each with independent limits. The solution here is a custom async connection pool backed by Python's `deque`, holding a set of live IB connections that workers check out, use, and return. You won't find this pattern documented anywhere online for IBKR. It was built from scratch.

☐ ARCHITECTURE

A `ConnectionPool` class manages up to 8 simultaneous IBKR connections. Each request checks out a connection from the deque, performs the download, and returns it. Connections that fail health checks are silently replaced.

☐ CONCURRENCY

Up to 32 async workers run simultaneously via `asyncio.Semaphore`. Tickers are batched and dispatched with `asyncio.gather()`. Failed tickers are retried up to 3 times with exponential backoff.

☐ RESILIENCE

Progress is persisted to a JSON file after each ticker. Runs can be interrupted and resumed without re-downloading completed tickers. Stale connections trigger automatic reconnect. Error codes from TWS are suppressed selectively (contract-not-found vs genuine errors).

// 2__PRICEDOWNLOADER.PY | CONNECTION POOL CORE (SIMPLIFIED)

```python
class ConnectionPool:
    def __init__(self, host, port, max_connections=8):
        self.connection_queue = deque()          # double-ended queue of live IB connections
        self.lock = asyncio.Lock()               # prevents concurrent pool mutations
        self.connection_health = {}               # tracks failure/success per connection

    async def get_connection(self):
        async with self.lock:
            while self.connection_queue:
                tag, ib = self.connection_queue.popleft()
                if ib.isConnected():
                    return tag, ib           # healthy connection, check it out
            return await self._create_new_connection()   # pool exhausted, open new

# 32 concurrent workers, each holding one checked-out connection
semaphore = asyncio.Semaphore(32)
tasks = [process_with_semaphore(ticker) for ticker in batch]
await asyncio.gather(*tasks, return_exceptions=True)
```

// 05. STAGE 3: ALPHA ENGINEERING

50+ Features Engineered Across All CPU Cores

`3__AlphaSensitivity.py` transforms raw OHLCV bars into a feature matrix for each ticker. It also pulls live market context directly from IBKR TWS (VIX, SPY, QQQ, IWM, DIA) to compute features that place each stock in context of the current regime. All ticker-level computation runs in a `ProcessPoolExecutor` that saturates every available CPU core.

FEATURE CATEGORIES

CATEGORY	FEATURES
Market Beta & Alpha	Rolling beta, correlation, alpha vs SPY, QQQ, IWM, DIA (60-day window, 30-day min)
VIX Regime	VIX level, VIX-of-VIX, VIX momentum, VIX z-score, VIX spike/trough detection, DVAMR probability
Kalman Filter	Kalman-smoothed price trend, residuals from trend, regime state estimates
Spectral / FFT	Dominant frequency, spectral entropy, cyclical component isolation via scipy.fft
Visibility Graph	Natural visibility graph metrics via ts2vg + networkx: degree distribution, clustering, path length
Distribution	Rolling skewness, kurtosis, entropy, Gaussian KDE density at current price
Stress Signals	EWM-smoothed open-to-low drawdown frequency, volume-ratio stress, multi-severity stress indicators
Classic Technical	Momentum, volatility normalization, price relative to rolling bands, gap analysis

☐ DVAMR PROBABILITY: A CUSTOM FEATURE

Dynamic Volatility Adjusted Mean Reversion Probability is a custom composite signal that produces a 0–1 probability score for mean reversion conditions. It combines:

1. **VIX REGIME**
   Current VIX vs. its own rolling z-score and VIX-of-VIX
2. **VOLATILITY NORMALIZATION**
   Stock's return deviation normalized by its own rolling volatility
3. **MARKET STRESS COMPOSITE**
   Multi-severity stress indicator (moderate / severe / extreme) weighted by volume ratio

☐ WHY VISIBILITY GRAPHS ON PRICE SERIES?

A Natural Visibility Graph (NVG) converts a price series into a graph: each bar is a node, and two nodes are connected if one can "see" the other without a taller bar in between. The resulting graph's topology (degree distribution, clustering coefficient, mean path length) captures structural properties that FFT and rolling statistics miss, things like trend persistence, local complexity, and dominant peak structure. These features come from the academic literature on graph-theoretic time series analysis and add a genuinely distinct information channel to the feature matrix.

// 06. STAGE 4: PREDICTION

XGBoost Binary Classifier with Isotonic Calibration

The predictor answers one question per ticker per day: will this stock close higher tomorrow than it closes today? That's it. The simplicity is deliberate, and it came from running experiments where making the target more complex made the model measurably worse.

Simple target outperforms complex target EMPIRICAL FINDING

An earlier version of this system tried to push trading rules directly into the prediction label: encoding position sizing, stop logic, and exit conditions so the model could learn better trading decisions in one shot. AUC-PR dropped consistently every time the label got richer. What was happening: a complex label increases label variance without adding signal. Two identical entry conditions can produce completely different outcomes depending on where the stop sits or how long the trade runs. That variance goes into the training labels as noise. A simple binary directional label has the lowest possible noise and the clearest relationship to the input features. Trading logic belongs in the strategy layer. The predictor just calls direction.

☐ MODEL CONFIGURATION

objective	binary:logistic
eval_metric	aucpr (precision-recall)
num_parallel_tree	24
nthread	32
max_depth	3 (intentionally shallow)
early_stopping_rounds	10
data splits	temporal with 5-day embargo gaps
calibration	isotonic (recency-weighted)

☐ WHY AUC-PR INSTEAD OF AUC-ROC?

Stocks go up slightly less than half the time after fees, so the training data is class-imbalanced. AUC-ROC is insensitive to class imbalance and scores well even for a model that performs poorly on the minority class. AUC-PR (area under precision-recall curve) specifically measures performance on positive cases. Early stopping on AUC-PR means the model stops when it stops improving precision on actual up-moves, which is the right objective for a trading signal.

☐ WHY ISOTONIC CALIBRATION?

Raw XGBoost scores are not probabilities. A score of 0.65 does not mean 65% chance of going up. Isotonic regression maps raw scores to empirically calibrated probabilities on a held-out set. A 2% confidence margin on top of the decision threshold cuts out marginal calls. The output is a number the backtester and execution engine can treat as an actual probability estimate rather than just a ranking signal.

// 07. STAGE 5: BACKTESTING

Research-Grade Backtesting with 20+ Metrics

The backtester is where strategy ideas get tested and broken before any real capital is at risk. It runs on Backtrader with an exact replica of the IBKR commission structure: $0.0035/share, $0.35 minimum, 1% value cap, plus exchange fees. Every metric prints to the terminal in color, graded from red to green. One exception: values so exceptional they qualify as "Unicorn" render in cornflower blue. That label comes from Van Tharp's SQN framework, which turned out to be a useful reference point for calibrating what a genuinely good result looks like.

PERFORMANCE METRICS TRACKED

METRIC	PURPOSE
Annualized Return %	Core performance, CAGR equivalent
Sharpe Ratio	Risk-adjusted return vs. daily vol
Sortino Ratio	Downside-deviation adjusted return
Calmar Ratio	Return / max drawdown
Omega Ratio	Probability-weighted gain vs. loss
Gain-to-Pain Ratio	Sum gains / sum losses
Max Drawdown %	Worst peak-to-trough decline
Max DD Duration	Days underwater
Ulcer Index	RMS of drawdown depth over time
Recovery Factor	Net profit / max drawdown
VaR 95% / CVaR 95%	Tail risk, daily
Annualized Volatility	Daily vol × √252
Win Rate % (after fees)	Profitable trades / total
Avg Win / Avg Loss	Payoff structure
Risk of Ruin	Probability of catastrophic loss
VWR	Variability-Weighted Return
Common Sense Ratio	Profit factor adjusted for tail
SQN (modified)	System Quality Number, see scale at right

SQN QUALITY SCALE (COLORIZED)

The System Quality Number measures mean trade return divided by its standard deviation, scaled by the square root of trade count. Every metric in the terminal maps to a continuous red-to-green gradient. SQN values above 15 get a separate cornflower blue designation called Unicorn. It comes up rarely enough that when it shows up, it actually means something.

● Negative	SQN < 0
● Poor	0 – 1.6
● Below Average	1.6 – 2.0
● Average	2.0 – 2.5
● Good	2.5 – 3.0
● Excellent	3.0 – 5.0
● Superb	5.0 – 7.0
● Holy Grail Potential	7.0+
● Unicorn ✦	SQN ≥ 15 (cornflower blue)

☐ DYNAMIC THRESHOLDS: CONTEXT-AWARE COLORIZATION

Fixed red/green thresholds on every metric would be misleading. A 150%+ annualized-return strategy will have higher volatility than a 20% strategy, and that's fine. The backtester computes acceptable volatility bands, VaR bounds, and drawdown limits at runtime based on Sharpe, Sortino, and raw return. Color coding is always relative to what the specific strategy is actually doing, so a high-return system doesn't look broken just because it has numbers that would be bad in a different context.

// 08. STAGE 6: LLM MACRO FILTER

Claude in Research Mode as a Soft Filter

Quantitative signals have a blind spot for catastrophic binary events. A company filing for bankruptcy, announcing an emergency merger, or caught in a geopolitical freeze looks completely normal in the feature matrix right up until it doesn't. Price history can't predict these. Regex can't catch them reliably. This stage uses Claude (Sonnet) in research mode to scan recent news on each signal ticker before the day's orders go out.

☐ WHAT THE LLM CHECKS

The model is prompted to search recent news (last 14 days for news, up to 60 days for M&A) and flag any of the following:

- Active or announced M&A (has_active_ma)
- Bankruptcy filing or imminent insolvency
- Geopolitical event affecting the company directly
- Regulatory action, trading halt, or exchange delisting
- Material crisis event (fraud, criminal investigation, etc.)

☐ ENGINEERING AROUND THE LLM

The LLM is not trusted for quantitative judgement. Its job is to catch qualitative flags a human analyst would spot in two minutes of reading headlines. Several engineering decisions keep this practical:

- Per-ticker results cached to JSON; interrupted runs resume without re-querying
- Errored tickers (rate limit, network) tracked separately and retried on resume
- `--skip-claude` flag bypasses LLM for cheapest/fastest run
- Hardcoded quantitative pre-filters run first (min price $2.00, min volume 206k)

// 09. STAGE 7: LIVE EXECUTION

Four-Legged OCA Brackets on a Personal Computer

The execution engine runs on a personal computer. Network outages happen. Power goes out. Open positions need protection even when the local software is completely offline. The approach is to submit all exit orders as server-side IBKR orders grouped in an OCA (One-Cancels-All) group at the time of entry. Once transmitted, IBKR holds them. The local machine can go dark and the downside protection stays live at the exchange.

Four-legged bracket structure OCA GROUP

Every position entry generates four orders submitted as a single atomic bracket. All three exit legs share one OCA group name (`bracket_{symbol}_{parent_id}`). The moment any exit fills at the exchange, IBKR cancels the remaining two. The local Python process doesn't need to be running. This is how the system works safely on consumer hardware.

☐ LEG 1: PARENT ENTRY (IBKR ADAPTIVE ALGO)

A limit order using IBKR's Adaptive algorithm at Normal priority. The Adaptive algo submits inside the spread and walks toward the ask over time, so fills land at mid-price or better on liquid names. Expected slippage drops from 10-15 bps with a plain limit-at-ask down to around 2-5 bps. The limit price is a safety ceiling, not the expected fill price. Slippage at each fill is logged to CSV to build a running empirical record.

☐ LEG 2: TAKE-PROFIT (LMT, GTC)

A limit sell at the backtester-derived target price, or at entry + 2× risk if no target is provided. Good-till-cancelled and valid outside regular hours so after-hours announcements trigger the exit. OCA type 1: fills this, cancels the trailing stop and hard stop.

☐ LEG 3: TRAILING STOP (TRAIL, GTC)

A percentage-based trailing stop computed from ATR at signal time. Floor 1.2%, ceiling 4.0%. These are intentionally tighter than the backtester (1.5%/5.0%) to give back less from peak in live trading. Stays server-side, rises with the price, never falls.

☐ LEG 4: HARD STOP (LMT, GTC)

A fixed limit order below the trailing floor. Covers the case where the trailing stop gets gapped over during a halt or an after-hours move. Sits in the same OCA group and is tighter than the trailing floor, so it only fires when something genuinely unexpected happens.

☐ ENTRY TIMING (DERIVED FROM EDA)

The 10:00 ET entry time came from an EDA study on 1,231 signal-day observations. Buy-at-open vs buy-at-10:00, exit at close:

Entry time	Sharpe	Win Rate
Buy at 9:30 open	0.31	49.4%
Buy at 10:00 ET	1.11	50.9%

+256% Sharpe improvement from waiting 30 minutes.

☐ SPY DIRECTION GATE (ALSO EDA-DERIVED)

At 10:00 ET the system checks SPY's move from open. If SPY is down more than 0.5%, all orders are cancelled for the day. From the same EDA dataset:

SPY condition	Sharpe	Win Rate
SPY < -0.5%	-6.50	31.5%
SPY green day	+5.34	62.0%
SPY > +0.5%	+6.77	68.2%

// 10. DESIGN DECISIONS

Key Engineering and Research Choices

Several decisions in this system are non-obvious. This section explains the ones that come up most often when walking someone through the codebase.

Temporal data splits with 5-day embargo gaps ML

Standard train/test splits chosen at random introduce lookahead bias in financial time series. A row from 2022 in the test set alongside a row from the same week in training means both share the same market regime, making the split artificially easy. This system uses strictly temporal splits where all training data precedes all test data, with a 5-day embargo gap between them. The embargo prevents the model from learning on data that is temporally adjacent to, and therefore correlated with, the test period. This approach comes from *Advances in Financial Machine Learning* (Lopez de Prado) and is standard practice in serious quantitative ML.

Why not more complex targets for XGBoost? ARCHITECTURE

When the prediction target was enriched to encode exit rules, sizing logic, and multi-day holding behavior, measured AUC-PR dropped consistently across experiments. The explanation is that a complex target increases label noise without increasing signal: the variance in realized outcomes from identical entry conditions (due to stop placement, holding period, market microstructure) adds irreducible noise to the training labels. A simple binary directional target has the lowest possible label noise and the clearest relationship to the input features. Complexity belongs in the strategy layer, not the prediction layer.

Zero-slippage assumption is defensible EXECUTION

Backtests with zero slippage get dismissed as unrealistic. For this system the assumption holds up: the IBKR Adaptive Algo fills limit orders at mid-price or better on liquid names. Empirical slippage logging from live trading confirms fills landing at 2-5 bps, well inside the noise floor of daily return variance. Commissions are modeled exactly ($0.0035/share, $0.35 minimum), and fills are treated as occurring at the signal price. That is consistent with what the execution data actually shows.

Why exclude banks, REITs, ETFs? UNIVERSE

OHLCV-derived alpha factors work well for industrial, technology, consumer, and healthcare companies. Banks are a different problem: their returns correlate more with credit spreads and rate regimes than with the price-based features this system measures, and their balance sheets are opaque enough to make standard signals unreliable. REITs are driven by distribution yield dynamics. ETFs add index arbitrage and replication mechanics to the mix. Removing all of these concentrates the model's training signal where the features are actually informative.

Recency-weighted isotonic calibration ML

Standard isotonic calibration weights all calibration samples equally. This system applies recency weighting so recent observations carry more influence. Market regimes shift. A calibration curve fit on 2019 data produces miscalibrated probabilities in 2024. Recency weighting keeps the probability outputs tracking the current relationship between raw scores and realized outcomes rather than a stale historical average.

// 11. ACADEMIC FOUNDATIONS

The Research That Shaped This System

Five books contributed specific techniques that appear directly in the code. This section covers what each one actually contributed rather than listing them as general influences.

01 **Advances in Active Portfolio Management**
Richard C. Grinold & Ronald N. Kahn

The IC framework for scoring alpha factors, the Fundamental Law of Active Management, and the concept of breadth as independent bets all fed directly into the alpha factor engineering stage. The asset correlation work in `0__AssetCorrolator.py` draws from the covariance estimation methods in this book specifically.

02 **Machine Learning for Algorithmic Trading**
Stefan Jansen

The feature engineering approach (rolling windows, lag features, cross-sectional normalization) follows patterns from this book. The AUC-PR evaluation methodology and the framing of prediction as a ranking problem rather than a regression problem both appear directly in this system. Practically focused and worth reading before touching any financial ML work.

03 **Advances in Financial Machine Learning**
Marcos Lopez de Prado

The source of the temporal split methodology with embargo gaps used in this system. De Prado's demonstration that random cross-validation is inappropriate for financial time series, and that walk-forward splits with purging and embargo are the correct approach, directly motivated the training pipeline. The meta-labeling discussion (predicting trade quality rather than just direction) also influenced the decision to keep the prediction target simple.

04 **Dynamic Hedging: Managing Vanilla and Exotic Options**
Nassim Nicholas Taleb

No options in this system. The book contributed anyway. Taleb's treatment of tail risk and the inadequacy of variance as a complete risk description informed the decision to track Ulcer Index, CVaR, and Risk of Ruin in the backtester alongside standard volatility measures. The priority on downside protection (bracket orders, hard stops, server-side exits) comes from that same orientation.

05 **Assessing and Improving Prediction and Classification**
Timothy Masters

Probably the least known book on this list and worth tracking down. Masters covers how to avoid fooling yourself with in-sample performance, how to design robust out-of-sample tests, and how calibration affects the practical usefulness of a probabilistic predictor. That material directly shaped the AUC-PR methodology, the calibration stage, and the confidence margin on the decision threshold. His approach of treating prediction quality as multi-dimensional rather than a single number also influenced the metrics system in the backtester.

Jonathan Bellmont
masamunex9000@gmail.com
github.com/JonIsHere242 | github.com/JonIsHere242/Stock-Market-Production
Autonomous Trading System | End-to-End Quantitative Pipeline

LANGUAGES & INFRASTRUCTURE
`Python 3.11+` `PowerShell`

MARKET DATA & EXECUTION
`ib_insync` `backtrader` `backtrader_ib_insync` `yfinance` `exchange_calendars` `pandas_market_calendars`

DATA ENGINEERING
`NumPy` `pandas` `pyarrow` `joblib` `pytz`

MACHINE LEARNING
`XGBoost` `scikit-learn` `Numba`

SIGNAL PROCESSING & FEATURE ENGINEERING
`scipy` `pykalman` `ts2vg` `networkx`

VISUALIZATION
`Plotly` `Matplotlib`

ASYNC & NETWORKING
`asyncio` `nest_asyncio` `requests` `urllib3` `watchdog`

AI RESEARCH LAYER
`Anthropic SDK` `tqdm`